

■# Product Description

Platform Version: 0.0.4

Author: Meow Li

Copyright: Copyright (c) 2026 Meow Li. All Rights Reserved.

One-Sentence Summary

Partner is a bridge bidding platform where players define their partnership agreements, compile those agreements into an explainable bot partner, and test those systems in human-plus-custom-bot competition.

What The Product Is

Partner is not meant to be another fixed robot bridge table. The central idea is that each player can bring their own bidding system.

Instead of playing with a generic bot that guesses what you mean, you define what your partnership plays. The platform turns those agreements into a bot partner that tries to follow them strictly and explainably.

Partner should be understood as a layered product, not a single monolithic feature.

Layer 1 is the **custom convention/system editing and execution system**. This layer lets users define systems, author gadgets, import and export system files, generate human-readable system notes, and execute the system through an explainable bidding engine. It also includes planned LLM workflows that convert natural-language system notes into structured system and gadget files.

Layer 2 is the **tournament and inter-user platform**. This layer uses the systems from Layer 1 to support play, comparison, sharing, competition, system-vs-system testing, and human-plus-custom-bot events.

The layers should be designed separately. The system/gadget engine must be useful even before the tournament platform exists. The tournament platform should depend on the system engine rather than mixing gameplay, accounts, sharing, and bidding logic into one inseparable unit.

The authoring layer should eventually support several ways to describe the same agreement:

- ? guided GUI forms for common bridge structures,
- ? a formal Bridge System Language for readable expert authoring,
- ? direct IR/YAML inspection or editing for advanced technical users.

All paths must compile or validate into the same strict executable representation before the bidding engine uses them.

The core game model is:

Human + Custom Bot Partner
vs
Human + Custom Bot Partner

Two human users can compete without needing four people online at the same time. Each user's bot partner bids according to that user's declared system.

The Problem

Bridge players often care deeply about partnership agreements. A common frustration with robot bridge is not that the robot plays badly in a general sense, but that it does not play **your** agreements.

Examples:

- ? You play four-way transfers, but the robot treats a bid as natural.
- ? You play a double as penalty, but the robot pulls it.
- ? You play an artificial raise, but the robot explains it vaguely.
- ? You want to test whether Bergen raises or inverted minors improve your results, but generic robot play blurs the experiment.
- ? You and a partner have a detailed system, but it is hard to test without arranging live practice.

Partner is designed around the idea that the system should be explicit, testable, shareable, and explainable.

Core Value Proposition

Layered Platform

The product has two major modules.

System Authoring And Execution Module

This module is the foundation.

It should support:

- ? Editing bidding systems.
- ? Editing portable gadgets.
- ? Sharing system and gadget files.
- ? Importing and exporting system definitions.
- ? Dealing hands, entering hands manually, and importing deals from formats such as PBN.
- ? Running bidding practice or system tests on imported deals.
- ? Generating human-readable system notes and reports.
- ? Converting human-readable system notes into structured files with LLM assistance.
- ? Running the bidding engine against a hand and auction.
- ? Explaining public meaning and internal origin.

This module can be valuable by itself as a training, system design, and partnership-agreement tool.

Tournament And Inter-User Module

This module comes after the system engine is reliable.

It should support:

- ? Human-plus-custom-bot versus human-plus-custom-bot play.
- ? User accounts and saved systems.
- ? System sharing, copying, and forking.
- ? Inter-user challenges.
- ? Tournament-style comparison on shared boards.
- ? Analytics about system performance and agreement gaps.

This module should treat systems and gadgets as imported artifacts from the authoring/execution layer.

Customizable Partnership Agreements

Users can define what their bot partner plays. Casual users should eventually do this with preset controls. Advanced users should be able to define exact auctions and custom gadgets.

In user-facing language, a **basic system** or **base system** means the foundational agreement set that handles openings, basic responses, and early auction structure. In the technical engine, the foundational agreement set is represented as a normal gadget.

Example casual choices:

- ? 2/1 game forcing.
- ? Four-way transfers.
- ? Bergen raises.
- ? Inverted minors.
- ? Puppet Stayman.
- ? Splinters.
- ? Competitive doubles.

Example advanced choices:

- ? What does `X` mean after `1H 1S 4H`?
- ? Is a double penalty or takeout?
- ? What continuations apply after an artificial relay?
- ? When may partner pull a penalty double?
- ? What counts as a superaccept hand?

Strict Agreement Following

The bot should follow the selected system. It should not silently replace the user's agreement with generic bridge culture.

If the system says partner must pass a penalty double unless an explicit escape condition applies, the bot should respect that.

Explainable Bidding

Every selected bid should be traceable.

The platform should be able to answer:

- ? Which gadget produced this bid?
- ? Which rule inside that gadget applied?
- ? What public meaning does the bid have?
- ? What other candidate bids were compared?
- ? What structured criteria caused the chosen bid to win?
- ? Was the call alertable?

There are two explanation layers:

1. **Public meaning**: what opponents or a convention card should know.
2. **Internal origin**: training and debugging details, including candidate comparisons.

For example, after:

1NP2DP

with:

S AQ74 / H KJ83 / D A62 / C Q5

the current prototype selects:

3H

The public meaning says it is a superaccept of the heart transfer. The internal origin records that `3H` beat `2H` because the pending heart transfer existed, the hand had maximum-ish notrump values, and there was four-card heart support.

Shareable Systems

Users should eventually export, import, fork, and share bidding systems.

Possible workflows:

- ? A teacher publishes a beginner 2/1 system.
- ? A pair forks that system and adds their own competitive doubles.
- ? A club shares a recommended convention set.
- ? A professional player sells or licenses a premium system file.
- ? A user exports a gadget file for four-way transfers and another user imports it into a different system.
- ? A user generates a system-notes report from structured gadget files.
- ? A user uploads human-readable notes and asks the LLM-assisted tool to draft matching system/gadget files.

Competitive Testing

The platform should let users test systems against each other on the same boards.

This supports questions like:

- ? Does my notrump structure perform better with four-way transfers?
- ? Is my competitive double structure robust?
- ? Does this gadget create too many undefined continuations?
- ? Do my agreements produce better auctions than a standard preset?

Target Users

Casual And Intermediate Bridge Players

These users should not need to write code or formal rule files.

They should be able to choose agreements through:

- ? Presets.
- ? Checkboxes.
- ? Dropdowns.
- ? Convention-card-like controls.
- ? Guided questions.

Example:

```
System base: 2/1 Game Forcing
Notrump range: 15-17
Transfers: Four-way transfers
Major raises: Bergen raises
Minor raises: Inverted minors
Splinters: On
```

Advanced Players

Advanced players want control over exact auctions.

They may want to define:

- ? System-specific doubles.
- ? Forcing pass auctions.
- ? Artificial continuations.
- ? Superaccept rules.
- ? Partnership obligations.
- ? Custom hand-evaluation criteria.

Teachers And Coaches

Teachers can use the platform to:

- ? Demonstrate why an agreement exists.
- ? Show what a bid means.
- ? Compare candidate calls.
- ? Assign practice auctions.
- ? Detect missing continuations in a student's system.

System Designers

System designers can use the platform as a laboratory for bidding methods.

They can build gadgets, run tests, and publish systems for other users.

Potential Investors And Partners

The product has potential as:

- ? A training platform.
- ? A competitive bridge product.
- ? A system-sharing marketplace.
- ? A tool for teachers and clubs.
- ? A premium advanced-system builder.

What Is A Gadget?

A gadget is a portable bidding module.

Examples:

- ? Four-way transfers.
- ? Bergen raises.
- ? Inverted minors.
- ? Puppet Stayman.
- ? Splinters.
- ? Competitive doubles.

A gadget is not just a single bid. It can include:

- ? Opening or response rules.
- ? Meanings of calls.
- ? Continuations.
- ? Selection criteria.
- ? Semantic facts.
- ? Alertability information.

- ? Public explanations.
- ? Internal training/debug information.

For example, a four-way transfer gadget may include:

- ? Responder's transfer bid.
- ? Opener's normal completion.
- ? Opener's superaccept.
- ? Responder's continuations.
- ? Invitational sequences.
- ? Game-forcing sequences.

Product Principles

Agreement-Driven, Not Opaque

The platform should not primarily choose bids through opaque simulation. Evaluation is allowed, but it should be defined by the selected system or gadget.

If a gadget uses a phrase like "good suit," that should eventually be backed by a formal metric or predicate.

Semantic State Is Derived

The client should submit the visible bridge situation:

- ? System.
- ? Seat.
- ? Auction.
- ? Hand.
- ? Dealer.
- ? Vulnerability.
- ? Scoring.

The client should not submit hidden engine state like "transfer pending." The engine reconstructs that by replaying the auction.

Modular UI, Parallel Logic

The user interface may show active gadgets as a rack of cards or modules. But the engine is not a linear pipeline.

Correct mental model:

```
Auction + Hand + Environment + Active Gadgets
-> applicable rules
-> candidate calls
-> selected call
```

This means a base 2/1 gadget can define the `1N` opening, while a separate four-way Jacoby transfer gadget defines `1NP` responses. The engine should search all active gadgets before falling back to default behavior.

Example Product Scenario

Alice wants to test her notrump structure.

She chooses:

- ? 2/1 base system.
- ? 1N opening range 15-17.
- ? Four-way transfers.
- ? Superaccepts enabled.

The auction begins:

1NP2DP

Alice's bot holds:

S AQ74 / H KJ83 / D A62 / C Q5

The platform recommends:

3H

Alice can inspect:

- ? Public meaning: superaccepting a heart transfer.
- ? Origin: four-way transfers gadget, `superaccept\_hearts` rule.
- ? Compared candidates: `3H` and `2H`.
- ? Structured reasons: pending heart transfer, 16 HCP, four-card support.

Planned Product Areas

Users choose common agreements through guided UI controls.

Advanced users can write portable gadgets with formal rules.

The platform should eventually let users describe a gadget in ordinary bridge language, then use an LLM-assisted conversion workflow to draft Bridge System Language or structured rule files.

Example user request:

Create a gadget for four-way transfers over 1N. 2D transfers to hearts, 2H transfers to spades, opener can complete normally or superaccept with four-card support and maximum values.

The LLM feature should not silently install unverified rules. It should produce a draft gadget that the user can inspect, test, edit, and approve. The platform should show the generated BSL when available, the compiled IR/YAML, public meanings, alertability

assumptions, semantic state effects, and selection criteria before the gadget becomes active.

The system should warn users about:

- ? Undefined auctions.
- ? Conflicting meanings.
- ? Missing continuations.
- ? Ambiguous gadget overlaps.
- ? Unspecified partner obligations.

Example:

Warning: 1H 1S 4H X is undefined in this system.

For now, each rule can include an `alertable` field. Eventually, the platform should analyze whether a call is alertable under the relevant ACBL rules version.

Client-Side Training Hints

Some or all system logic could run client-side so the platform can hint what a user should bid during training.

Hand And Deal Workspace

The system authoring layer should include a workspace for hands, deals, and auctions. Users should be able to enter a hand manually, deal random hands, import full deals from formats such as PBN, select a system, and step through the bidding with engine explanations.

This component is not a tournament feature. It belongs to the system authoring and execution layer because it helps users test agreements, prepare lessons, analyze example deals, and debug gadgets before using them in inter-user play.

Business Model Ideas

Possible tiers:

Free

- ? Preset systems.
- ? Simple gadget toggles.
- ? Casual play.
- ? Limited sharing.

Paid

- ? Advanced system builder.
- ? Custom gadgets.
- ? System analytics.
- ? More sharing features.

? Tournament access.

Pro

? Full advanced rule access.

? Custom evaluation functions.

? Versioned system repositories.

? Advanced competitive agreements.

? High-level tournament modes.

These are product ideas, not final pricing decisions.

Current Prototype Snapshot

The current checkpoint is a Python backend prototype with canonical bidding notation, directory-based gadget files, a starter 2/1 gadget, a starter four-way Jacoby transfer gadget, auction replay through public meanings, semantic facts, candidate comparison, and separate public/internal explanation output.

Open implementation work and future product ideas are tracked in ``docs/todo.md`` instead of being maintained as a long backlog in this product overview.